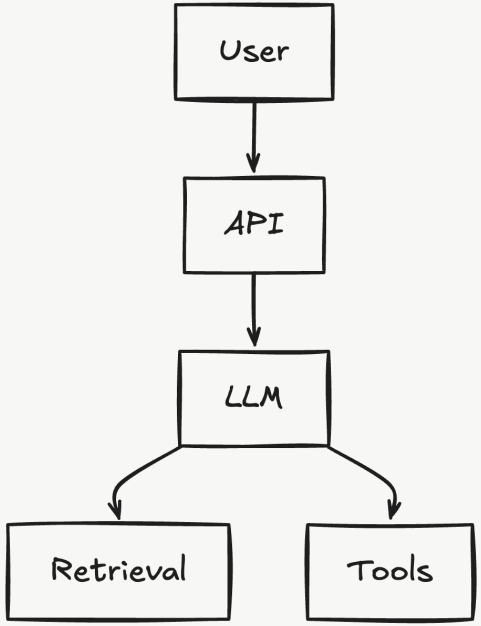
AI Systems Are Distributed Systems Now

Tracing the path, measuring the system,
and evaluating the outcome.

OBSERVABILITY × DISTRIBUTED SYSTEMS

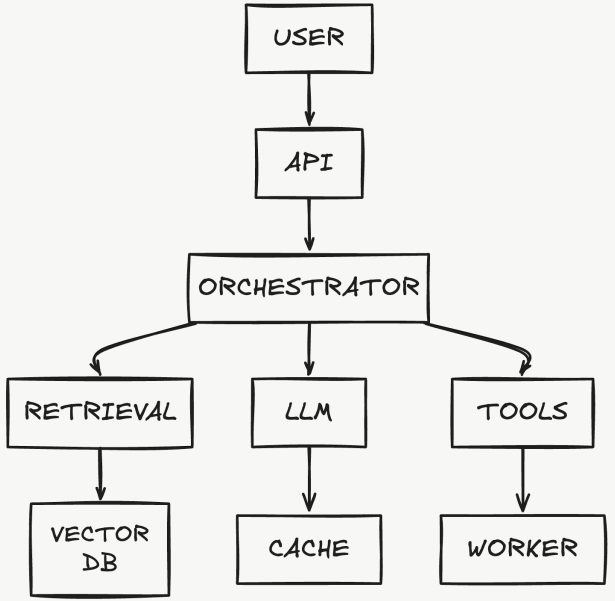


one request → many boundaries → many ways to fail

The model is no longer the application

Old mental model

It is one dependency inside a distributed execution graph.

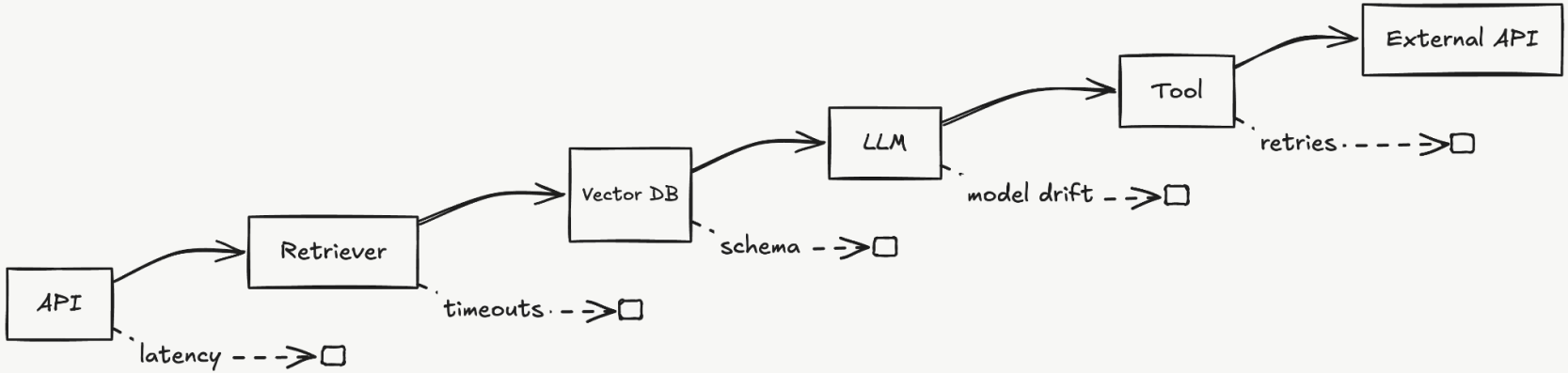


Modern AI application

Each boundary adds latency, state, failure, retry, and versioning concerns.

Instrument the boundaries

Distributed-systems failures often appear between components, not inside them.

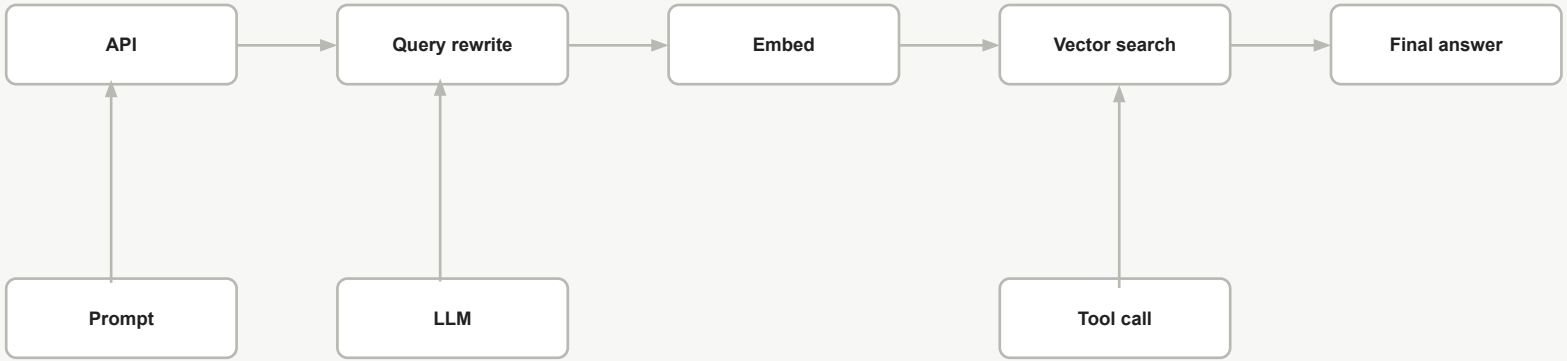


A good trace makes each hop explicit and keeps one request identity across the whole path.

TRACE ID

A single answer is the product of a chain of decisions

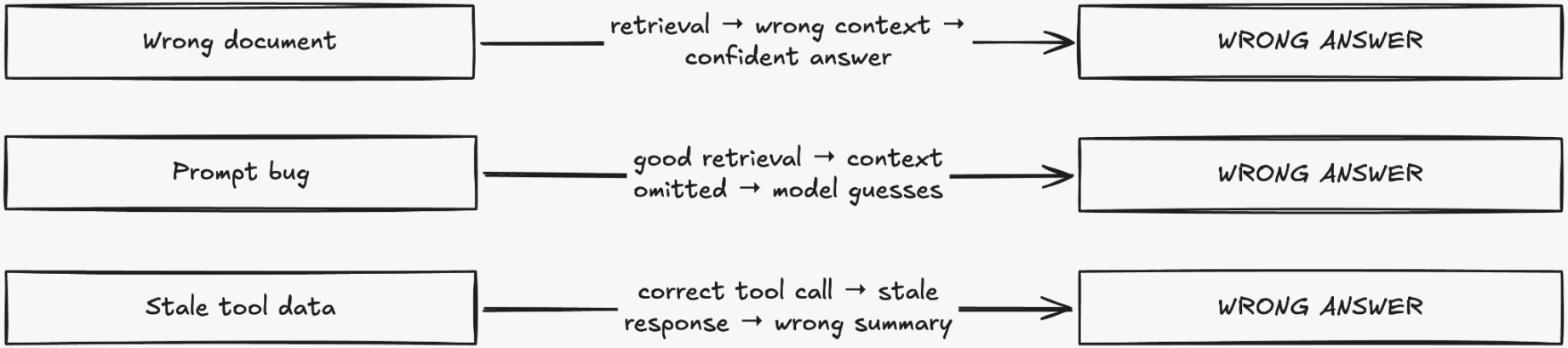
The output is the last step in a distributed execution graph.



Question → context → decision → action → answer

The model output is often the symptom, not the root cause

Same user-visible failure. Completely different root causes.



Without the trace, every failure looks like “the model hallucinated.”

Trace the decision path, not just the request path

Your trace should explain why the system arrived at this answer.

What to capture

- trace / span identity
- prompt + prompt version
- retrieved documents
- model + parameters
- tool call + result
- tokens + latency
- evaluation score

Traditional telemetry can say “healthy” while the AI is wrong

Technical success and semantic success are different things.

SYSTEM TELEMETRY

Latency	**1.8 s**
Error rate	**0.2%**
CPU	**43%**
Availability	**99.99%**

HEALTHY

AI QUALITY

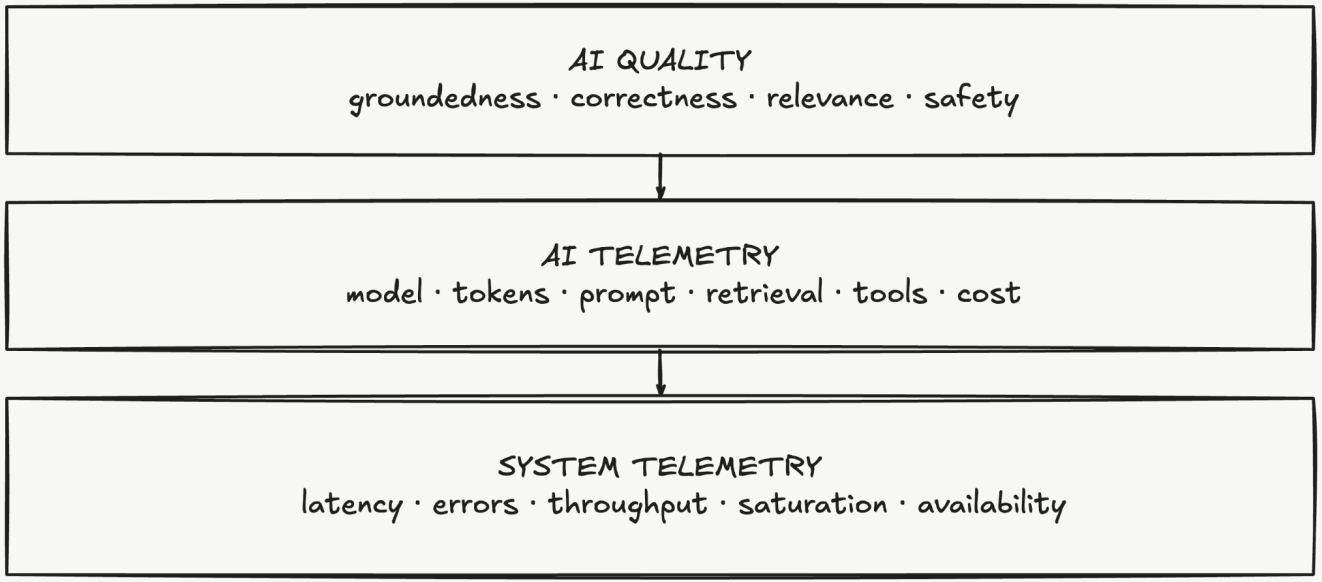
Groundedness	**0.42**
Correctness	**0.38**
Relevance	**0.91**
Safety	**0.97**

SEMANTIC FAILURE

The system is available. The answer is bad.

AI observability extends traditional observability

Do not replace infrastructure telemetry. Add the semantic layer above it.



Observe at the level of the system, the model, and the outcome.

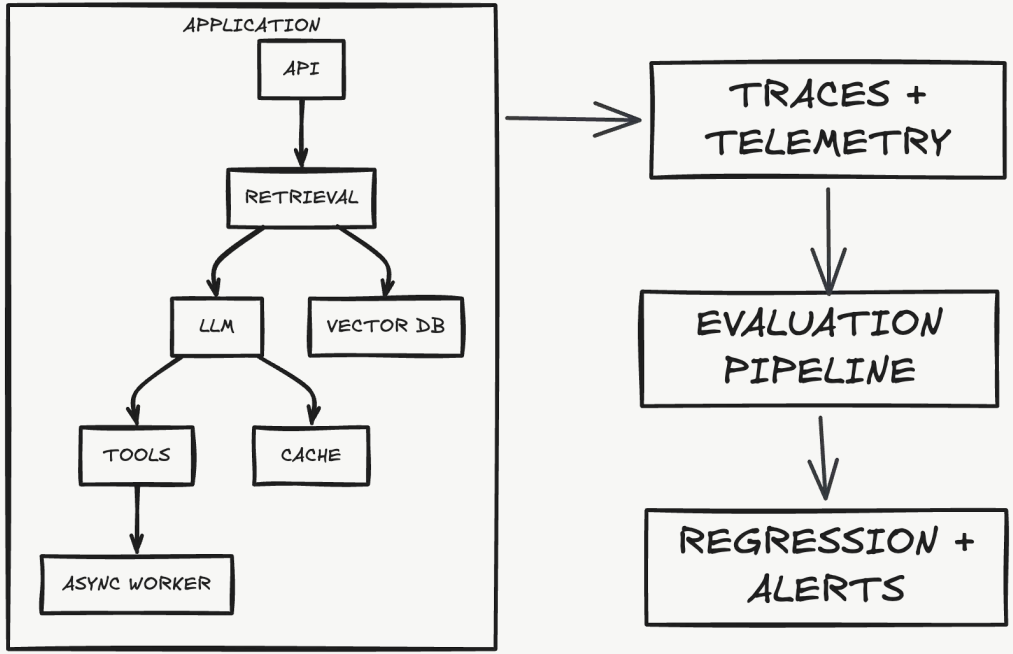
Observability asks “what happened?” Evaluation asks “was it good?”

This is the missing signal in traditional infrastructure telemetry.

Groundedness	Does the answer follow the supplied context?
Relevance	Did it answer the actual question?
Correctness	Is the claim factually / operationally right?
Safety	Did it stay within the expected policy?

A practical architecture for observable AI

Keep the application, telemetry pipeline, and quality loop conceptually separate.



The rules I would design around

A compact checklist for building observability into an AI system.

01 **Preserve causality**

One request identity across services, tools, workers, and evaluation.

02 **Instrument boundaries**

Capture the handoffs where latency, retries, state, and failures appear.

03 **Trace the decision path**

Record retrieval, context, model, tool use, and output, not just timing.

04 **Separate system health from AI quality**

A 200 response can still be a bad answer.

05 **Evaluate continuously**

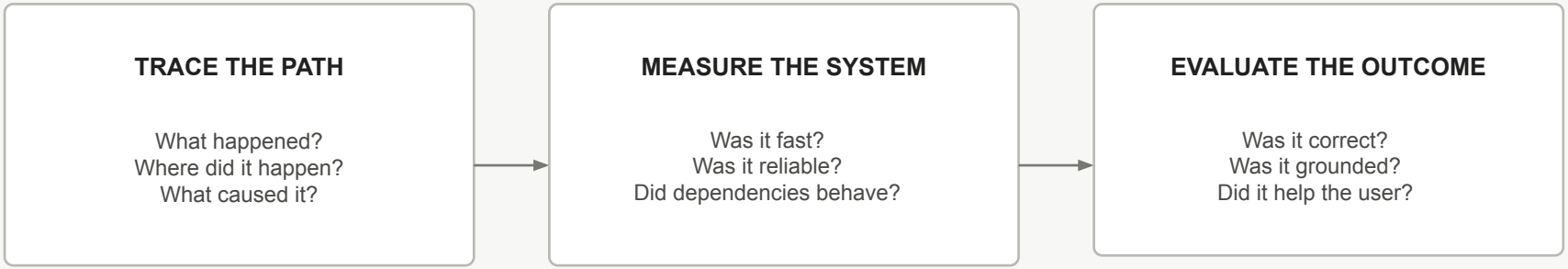
Turn traces into groundedness, correctness, relevance, and regression signals.

06 **Design telemetry as a system**

Sampling, redaction, retention, access control, and cost matter.

Trace the path. Measure the system. Evaluate the outcome.

That is the observability model for modern AI systems.



The model is only one part of the system.
The trace is how we understand the whole thing.